

PROYECTO DE RECUPERACIÓN (RA 4 – RA 6)

DESARROLLO DE UNA APLICACIÓN WEB CON SPRING BOOT - DATA

Se propone desarrollar una aplicación web para la gestión de los pedidos realizados en la cantina de un instituto.

El objetivo es aplicar los conocimientos adquiridos con el desarrollo de aplicaciones basadas en **Spring Boot**, el uso de **Spring Data JPA** para la persistencia de datos y la utilización de **Thymeleaf** para la generación de interfaces web.

La aplicación permitirá registrar alumnos, consultar los productos disponibles en la cantina y gestionar los pedidos realizados por los alumnos. Además, el sistema permitirá controlar el estado de los pedidos y gestionar el stock de los productos disponibles.

Este proyecto pretende reforzar conceptos importantes como:

- diseño de aplicaciones en arquitectura por capas
- modelado de datos con entidades JPA
- acceso a datos mediante repositorios
- implementación de lógica de negocio con servicios
- desarrollo de controladores web
- creación de interfaces mediante Thymeleaf
- validación de formularios utilizando Spring Validation

Objetivo

El objetivo principal del proyecto es desarrollar una aplicación web funcional que permita gestionar el funcionamiento básico de la cantina del instituto.

Los objetivos específicos del proyecto son los siguientes:

- Diseñar un modelo de datos adecuado para representar alumnos, productos y pedidos.
- Implementar entidades JPA que representen las tablas de la DB.
- Crear repositorios utilizando Spring Data JPA para gestionar el acceso a los datos.
- Implementar la lógica de negocio mediante servicios.
- Desarrollar controladores web que gestionen las peticiones HTTP.
- Crear páginas web dinámicas utilizando Thymeleaf.
- Controlar el stock de productos disponibles.
- Gestionar el estado de los pedidos realizados.

Descripción del sistema

La cantina del instituto ofrece diferentes productos como bocadillos, bebidas o snacks. Los alumnos pueden realizar pedidos durante el horario del recreo y el personal de la cantina debe gestionar dichos pedidos.

El sistema desarrollado permitirá realizar las siguientes operaciones:

- Consultar los alumnos registrados en el sistema.
- Consultar los productos disponibles en la cantina.
- Registrar nuevos pedidos realizados por los alumnos.
- Consultar los pedidos realizados por cada alumno.
- Consultar los pedidos pendientes de entrega.
- Marcar los pedidos como entregados.

Además, el sistema deberá comprobar que existe suficiente stock de un producto antes de permitir realizar un pedido.

Modelo de datos

La DB ha de llamarse “*gestion_cantina*” y el sistema estará compuesto por tres entidades principales:

- Alumno
- Producto
- Pedido

Estas entidades estarán relacionadas entre sí para representar el funcionamiento de la cantina.

Entidad Alumno

La entidad Alumno representa a los estudiantes que pueden realizar pedidos en la cantina.

Cada alumno tendrá almacenada su información básica.

Los atributos de esta entidad son:

- **id**: identificador único del alumno.
- **nombre**: nombre del alumno.
- **curso**: curso o grupo al que pertenece el alumno.
- **teléfono**: número de teléfono de contacto.

El identificador será generado automáticamente por la BD.

Entidad Producto

La entidad Producto representa los productos que se venden en la cantina.

Cada producto tendrá información sobre su precio y la cantidad disponible en stock.

Los atributos de esta entidad son:

- **id**: identificador único del producto.
- **nombre**: nombre del producto.
- **precio**: precio del producto.
- **stock**: cantidad disponible del producto en la cantina.

El sistema deberá controlar que el stock no sea negativo.

Entidad Pedido

La entidad **Pedido** representa los pedidos realizados por los alumnos.

Cada pedido estará asociado a un alumno y a un producto.

Los atributos de esta entidad son:

- **id**: identificador único del pedido.
- **alumno**: alumno que realiza el pedido.
- **producto**: producto solicitado.
- **cantidad**: número de unidades solicitadas.
- **fechaPedido**: fecha y hora en la que se realiza el pedido.
- **entregado**: indica si el pedido ya ha sido entregado o no.

Relaciones entre entidades

- Un alumno puede realizar varios pedidos.
- Un producto puede aparecer en varios pedidos.
- Cada pedido pertenece a un único alumno y a un único producto.

Estas relaciones deberán implementarse utilizando anotaciones **JPA** como @ManyToOne.

Funcionalidades del sistema

La aplicación deberá implementar las siguientes funcionalidades.

Gestión de alumnos

El sistema deberá permitir:

- Mostrar un listado con todos los alumnos registrados.

Esta funcionalidad permitirá visualizar los alumnos que pueden realizar pedidos en la cantina.

Gestión de productos

El sistema deberá permitir:

- Mostrar un listado de los productos disponibles en la cantina.
- Visualizar el precio y el stock de cada producto.

Gestión de pedidos

El sistema deberá permitir realizar las siguientes operaciones:

Registrar un pedido

Se deberá mostrar un formulario donde se pueda seleccionar:

- el alumno que realiza el pedido
- el producto solicitado
- la cantidad de unidades

Antes de registrar el pedido, el sistema deberá comprobar que el producto tiene suficiente stock disponible.

Si el stock es suficiente, el pedido se registrará y se actualizará el stock del producto.

Consultar pedidos de un alumno

El sistema deberá permitir visualizar todos los pedidos realizados por un alumno concreto.

Consultar pedidos pendientes

El sistema deberá permitir visualizar todos los pedidos que todavía no han sido entregados.

Marcar pedido como entregado

El sistema deberá permitir cambiar el estado de un pedido a **entregado** cuando el alumno reciba su pedido.

Arquitectura del proyecto

La aplicación deberá desarrollarse siguiendo una arquitectura basada en capas.

La estructura del proyecto será la siguiente:

- entity
- repository
- service
- controller
- templates

Capa Entity

Contendrá las clases que representan las entidades de la base de datos.

Capa Repository

Contendrá las interfaces que permiten acceder a la base de datos utilizando **Spring Data JPA**.

Capa Service

Contendrá la lógica de negocio de la aplicación.

Capa Controller

Gestionará las peticiones HTTP y conectará las vistas con la lógica de negocio.

Se deberán implementar controladores que gestionen las siguientes peticiones:

- **GET /alumnos**
→ Devuelve la lista de alumnos registrados.
- **GET /productos**
→ Devuelve la lista de productos disponibles en la cantina.
- **GET /pedidos/nuevo**
→ Muestra el formulario para registrar un nuevo pedido.
- **POST /pedidos/nuevo**
→ Registra un nuevo pedido si existe stock suficiente del producto solicitado.
- **GET /pedidos/alumno/{id}**
→ Devuelve la lista de pedidos realizados por un alumno.

- **GET /pedidos/pendientes**
→ Devuelve la lista de pedidos pendientes de entrega.
- **POST /pedidos/entregar/{id}**
→ Marca un pedido como entregado.

Templates

Contendrá las plantillas HTML desarrolladas con **Thymeleaf**.

Interfaces web

La aplicación deberá incluir al menos las siguientes páginas:

- Página de listado de alumnos.
- Página de listado de productos.
- Formulario para registrar un nuevo pedido.
- Página para consultar los pedidos de un alumno.
- Página para consultar los pedidos pendientes.
- Página para marcar pedidos como entregados.

Las páginas deberán desarrollarse utilizando Thymeleaf.

Se os proporcionará el esqueleto HTML básico para facilitar la creación de las vistas.

Validación de formularios

Los formularios deberán implementar validación utilizando **Spring Validation**.

Se deberán validar al menos:

- campos obligatorios en los formularios
- cantidades de pedido mayores que cero
- datos incorrectos o vacíos en la creación de pedidos

En caso de error, se deberá mostrar el mensaje correspondiente en la vista.

Requisitos técnicos

La aplicación deberá desarrollarse utilizando las siguientes tecnologías:

- Spring Boot
- Spring Data JPA
- Hibernate

- Thymeleaf
- Base de datos MySQL

Se deberán utilizar correctamente anotaciones como:

- @Entity
- @Id
- @GeneratedValue
- @ManyToOne
- @Repository
- @Service
- @Controller

9. Memoria del proyecto

Junto con el proyecto se deberá entregar una memoria en formato PDF que describa el desarrollo de la aplicación.

La memoria deberá incluir los siguientes apartados.

Introducción

Descripción general del proyecto y objetivos del sistema desarrollado.

Diseño del modelo de datos

Descripción de las entidades utilizadas y sus relaciones.

Se deberá incluir un diagrama entidad-relación.

Arquitectura del proyecto

Descripción de la estructura del proyecto y explicación de cada capa:

- entidades
- repositorios
- servicios
- controladores
- vistas

Implementación

Descripción de cómo se han desarrollado los principales componentes del sistema.

Se podrán incluir fragmentos de código explicativos.

Pruebas de funcionamiento

Se deberán incluir capturas de pantalla de la aplicación funcionando.

Ejemplos:

- listado de alumnos
- listado de productos
- registro de pedido
- pedidos pendientes
- pedido entregado

Conclusiones

Breve valoración personal sobre el desarrollo del proyecto y los conocimientos aplicados.

Entrega del proyecto

La entrega deberá incluir:

1. Proyecto completo de Spring Boot
 2. Memoria del proyecto en formato PDF
- El proyecto debe poder abrirse y ejecutarse correctamente en IntelliJ
 - Si al descomprimir y ejecutar se produce algún error no controlado → **suspenso**
 - Se os convocará en la semana de los exámenes ordinarios para la corrección individual del proyecto.